

Process Coordination

Dr. Jit Mukherjee



Introduction

- A cooperating Process is one that can affect or be affected by other processes
 - Either share a logical space, i.e. both code and data - Ex. Thread
 - Or share data through files or messages.
- Concurrent access to shared data -> Data Inconsistency
- Ensure orderly execution of cooperating processes such that data consistency is maintained.
- It is a procedure that is involved in order to preserve the appropriate order of execution of cooperative processes.
- In order to synchronize the processes, there are various synchronization mechanisms.
- Process Synchronization is mainly needed in a multi-process system when multiple processes are running together, and more than one processes try to gain access to the same shared resource or any data at the same time.

Race Condition

- Producer-Consumer Problem -> Bounded Buffer
 - Allows at most Buffer-Size - 1 items
 - Counter
 - Incremented to add new item
 - Decrement to remove one item
- Counter is currently at 3
 - Depending upon execution it can be 2, 3, 4
 - Correct result is counter = 3
- T_0 producer -> register1 = counter
- T_1 producer -> register1 = register1 + 1
- T_2 consumer -> register2 = counter
- T_3 consumer -> register2 = register2 - 1
- T_4 producer -> counter = register1
- T_5 consumer -> counter = register2

```
while {true}
{
    while (counter == Buffer_Size)
        /*do nothing*/
    buffer [in] = nextProduced;
    in = (in+1) % Buffer_Size;
    counter++;
}
```

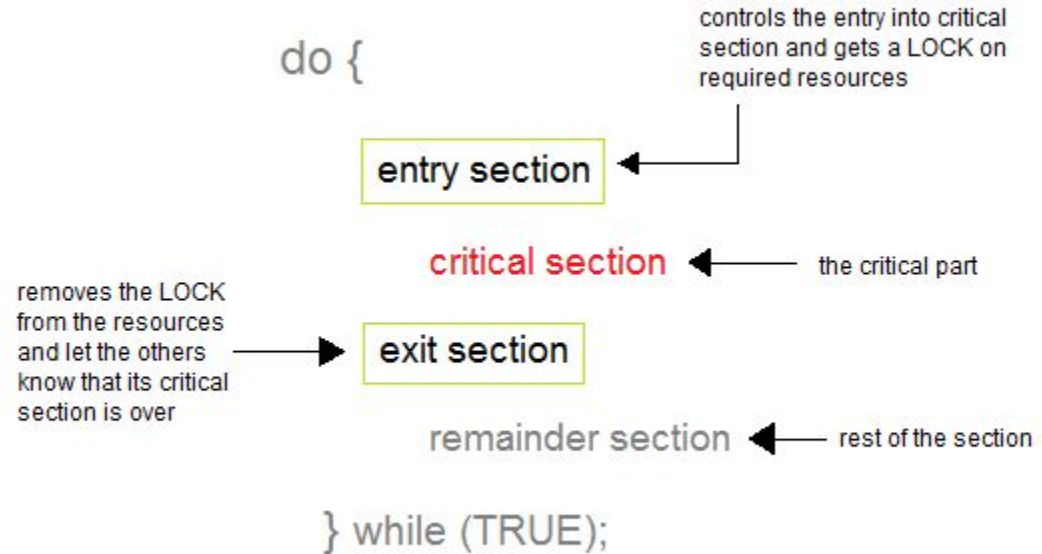
```
while{true}
{
    While (counter == 0)
        /*do nothing*/
    nextConsumed = buffer[out];
    out = (out+1) % Buffer_Size;
    counter--;
}
```

The Critical Section Problem

- Consider a system of n process $\{P_0, P_1 \dots P_{n-1}\}$.
- Each process has a segment of code, called critical section, in which the process may be changing
 - Common variable
 - Updating a table
 - Writing a file, etc.
- When one process is executing in its critical sections, no other process is to be allowed in it's critical section
 - No two process is executing their critical section
- The critical section problem is to design a protocol that the processes can use to cooperate.

Section of A Program

- **Entry Section:** It is part of the process which decides the entry of a particular process.
- **Critical Section:** This part allows one process to enter and modify the shared variable.
- **Exit Section:** Exit section allows the other process that are waiting in the Entry Section, to enter into the Critical Sections. It also checks that a process that finished its execution should be removed through this Section.
- **Remainder Section:** All other parts of the Code, which is not in Critical, Entry, and Exit Section, are known as the Remainder Section
- The entry to the critical section is mainly handled by **Wait()** function while the exit from the critical section is controlled by the **Signal()** function.



Rules for Critical Section

- **Mutual Exclusion (Safety):** If process P_i is executing in its critical section, then no other processes can be executing in their critical section.
- **Progress (Liveness):** This solution is used when no one is in the critical section, and someone wants to enter it's critical section.
 - Then only those processes not in their remainder section should decide who should go in.
 - in a finite time. The selection cannot be postponed indefinitely
- **Bound Waiting (Fairness):** When a process makes a request for getting into critical section, there is a specific limit about number of processes can get into their critical section. So, when the limit is reached, the system must allow request to the process to get into its critical section.
- **Architecture Neutrality** - If solution working for one architecture should work for others too. Non must condition

The Critical Section Problem Contd.

- Non Preemptive kernels is essentially free from race condition
 - A kernel-mode process will run until
 - It exists kernel
 - Voluntarily yield control of CPU
 - Windows XP
- Preemptive kernels are need to be carefully designed to avoid several race conditions
 - Maintaining memory allocation
 - File operation
 - Maintaining process list
 - Interrupt handling
- Preemptive kernel more suitable for real time programming.
 - After Linux 2.6

Peterson's Solution

- Classic software based solution
- Restricted to two process
 - Alternate execution between their critical and remainder section
- May not work correctly in modern architecture but provides a good algorithmic description of solving critical section problem
 - Assumption -> instructions are executed in a particular order and memory accesses can be achieved atomically.
 - On systems with multiple CPUs the algorithm may fail to work because the program order of events occurring on one CPU may be perceived differently on another.
- The process are numbered P₀ and P₁
 - When we refer P_i, P_j is denoted as the other process.
 - $j = 1 - i$
- Two data item
 - Int turn; (indicates whose turn to enter it's critical section)
 - Boolean flag[2]; (if a process ready to enter its critical section)

Peterson's Solution

- If flag [i] is true -> P_i is ready to enter it's critical section
- To enter critical section, process P_i first sets flag[i] -> true
 - turn -> j, means if the other process wishes to enter the critical section, it can do so
 - when a thread wants to enter the critical section, it must declare that the other thread gets to go first. Consequently, this update does not create a race condition.
- If both process try to enter at same time, turn will be set i and j. One will last
- Value of turn decides which process is allowed to enter in critical section

```
do {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn ==  
j);  
    /* critical section */  
    flag[i] = false;  
    /* remainder section */  
}  
  
while (true);
```

Peterson's Solution

1. Mutual exclusion is preserved.

- a. P_i enters its critical section only if either $flag[j] == false$ or $turn == i$.
- b. if both processes can be executing in their critical sections at the same time, then $flag[0] == flag[1] == true$.
- c. P_0 and P_1 could not have successfully executed their while statements at about the same time, since the value of $turn$ can be either 0 or 1 but cannot be both.
- d. one of the processes P_j must have successfully executed the while statement, whereas P_i had to execute at least one additional statement (" $turn == j$ ").
- e. $flag[j] == true$ and $turn == j$, and this condition will persist as long as P_j is in its critical section;

2. The progress requirement is satisfied.

3. The bounded-waiting requirement is met.

- a. P_i prevented to enter critical section $\rightarrow flag[j] = true$ and $turn = j$
 - i. P_j not ready to enter $flag[j] = false \rightarrow P_i$ enters
 - ii. P_j set $flag[j] = true$, $turn$ either i or j
 - 1. If $turn = i$, P_i enters
 - 2. If $turn = j$, P_j enters
 - iii. Once P_j exits critical $\rightarrow flag[j] = false \rightarrow P_i$ to enter
 - iv. If P_j resets $flag[j] = true \rightarrow turn = i \rightarrow P_i$ enters
- b. P_i will enter the critical section (progress) after at most one entry by P_j (bounded waiting).

Peterson's Solution

- This solution is also a busy waiting solution so CPU time is wasted. So that **"SPIN LOCK"** problem can come.
 - Spinlock is a lock that causes a thread trying to acquire it to simply wait in a loop ("spin") **while repeatedly checking whether the lock is available.**
- **Peterson's algorithm for N-Process** - Each process pushes itself in a queue to come at the end of the queue at some time
 - Then they will be allowed to execute their critical section and leave the queue.
- It doesn't know exactly how many processes are in the queue nor exactly what position in the queue it is in
- Two functions named lock() and unlock(). In the lock() function the process waits till it reaches the end of the queue and in unlock() function it marks the exit from the queue.
- The execution of the critical section takes place when the process is at the end of the queue i.e. between the lock and unlock methods.

```
LOCK( Process PID){
    for( int i = 0;i<N;++i){
        //Looping through position from 0 to end (N) of
queue.
        Turn[i]=PID;
        // Tells that PID process is at ith position.
        Flag[PID]=i;
        // Tells that till now there is no back push
        while((for all k != PID, Flag[k]<i) or (Turn[i]
!= PID));
        //Wait till either there is no process, at the
top of queue or there is a back push.
    }
}
UNLOCK(Process PID){
    Flag[PID]=-1
    //process exists or terminates or goes to another
queue
    else
    Flag[PID]=0
    // If process enter the first position at the queue.
}
```